



MÓDULO 3 · T10

Datos y API: Hooks Personalizados

 *Extrae lógica en hooks reutilizables*

Carlos Rodríguez Contreras · GitHub: `karrocon`

¿Qué aprenderás hoy?

En esta sesión aprenderás a extraer lógica compleja en hooks reutilizables y a combinar Context con hooks personalizados para gestionar el estado global de forma limpia y escalable.

1

Hooks personalizados

Entender qué es un hook personalizado y cuándo tiene sentido crearlo.

2

Fetch + estados

Extraer la lógica de fetch y sus estados en un hook reutilizable.

3

Context + hook

Combinar Context con un hook personalizado para gestionar estado global.

4

usePokemonsContext

Implementar el patrón completo con `usePokemonsContext` en la PokeApp.

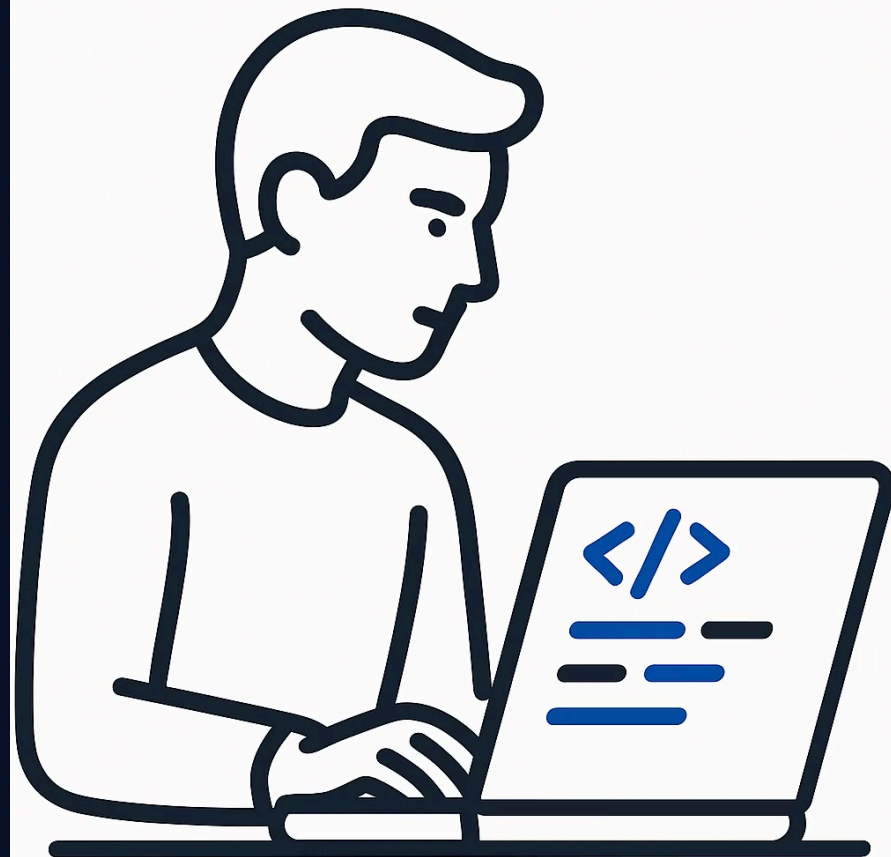
¿Qué es un hook personalizado?

Un hook personalizado es una función JavaScript cuyo nombre empieza por `use` y que puede llamar a otros hooks internamente. Permiten encapsular y reutilizar lógica stateful sin necesidad de duplicar código entre componentes.

```
// Hook personalizado
function useCounter(initial = 0) {
  const [count, setCount] = useState(initial);
  const inc = () => setCount(c => c + 1);
  const dec = () => setCount(c => c - 1);
  return { count, inc, dec };
}
```

```
// Uso en cualquier componente
const { count, inc, dec } = useCounter(10);
```

- ❗ La única regla es que el nombre empiece por `use` – así React puede aplicar las reglas de hooks y detectar usos incorrectos en tiempo de desarrollo.



Cuándo extraer un hook

✗ No extraer

No merece la pena crear un hook cuando la lógica es tan sencilla que añadir una abstracción solo aumenta la complejidad sin ningún beneficio real.

- Lógica simple de 2 líneas
- Solo se usa en un lugar
- Es solo presentación

⚠ Extraer demasiado pronto genera hooks innecesarios difíciles de mantener.

✓ Extraer

Crear un hook personalizado aporta valor cuando la lógica es compleja, se repite en varios sitios o mezcla efectos con estado.

- Lógica de fetch/estado compleja
- Se reutiliza en múltiples pantallas
- Mezcla de efectos + estado

✔ Un buen hook tiene una responsabilidad clara y un nombre que describe exactamente lo que hace.

De componente a hook

El refactor es directo: se mueve la lógica de estado y efectos fuera del componente a una función `useXxx`. El componente queda limpio y solo se ocupa de la presentación.

// Antes — lógica en el componente

```
function PokedexScreen() {  
  const [pokemons, setPokemons] = useState([]);  
  const [isLoading, setIsLoading] = useState(true);  
  useEffect(() => {  
    fetch(...).then(setPokemons);  
  }, []);  
  // ...  
}
```

// Después — lógica en el hook

```
function usePokemons() {  
  const [pokemons, setPokemons] = useState([]);  
  const [isLoading, setIsLoading] = useState(true);  
  useEffect(() => {  
    fetch(...).then(setPokemons);  
  }, []);  
  return { pokemons, isLoading };  
}
```

```
function PokedexScreen() {  
  const { pokemons, isLoading } = usePokemons();  
  // ...  
}
```

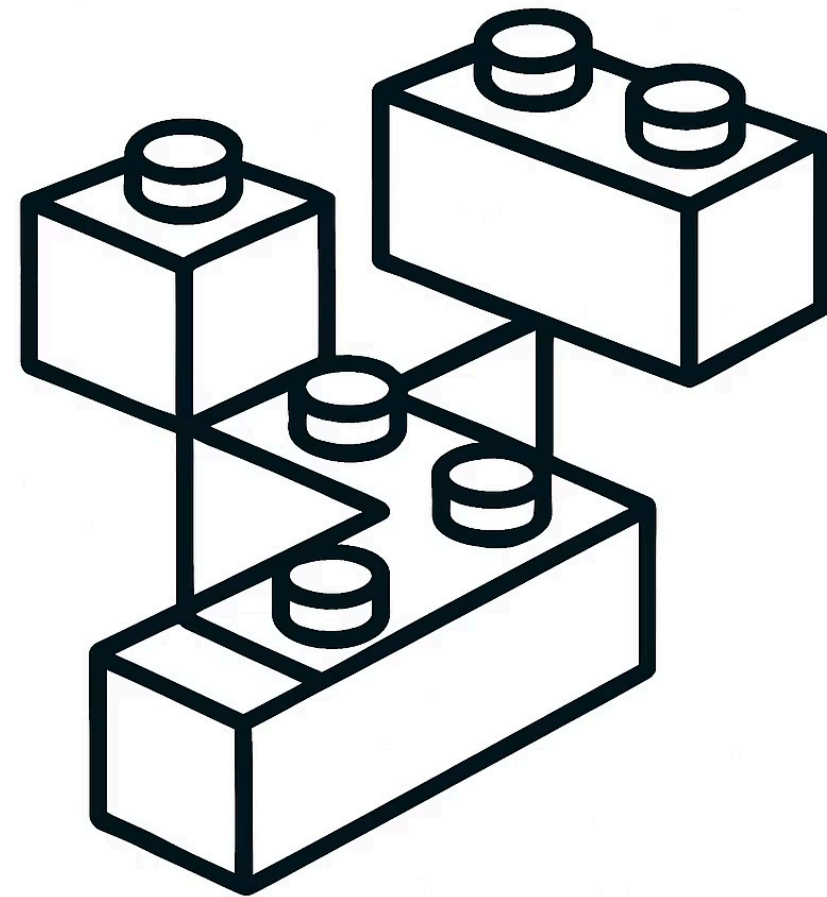
Context + hook = estado global

El patrón combina un `Context` para compartir datos con un hook personalizado que encapsula el acceso y añade validación. Cualquier componente del árbol puede consumir el estado con una sola línea.

```
// 1. Crear el contexto
const PokemonsContext =
  createContext<ContextValue | null>(null);

// 2. Provider con la lógica
export function PokemonProvider({ children }) {
  const [pokemons, setPokemons] = useState([]);
  // ... fetch, cache, etc.
  return (
    <PokemonsContext.Provider value={{ pokemons }}>
      {children}
    </PokemonsContext.Provider>
  );
}

// 3. Hook de acceso (con validación)
export function usePokemonsContext() {
  const ctx = useContext(PokemonsContext);
  if (!ctx) throw new Error('Fuera de PokemonProvider');
  return ctx;
}
```



PokeApp — usePokemonsContext.tsx

Esta es la implementación real del `PokemonProvider` en la PokeApp. Observa cómo `useRef` actúa como flag para evitar el doble fetch que React StrictMode provoca en desarrollo, y cómo el cache en `AsyncStorage` evita peticiones de red innecesarias entre sesiones.

```
export function PokemonProvider({ children }) {
  const fetchedRef = useRef(false); // evita doble fetch en StrictMode

  useEffect(() => {
    if (fetchedRef.current) return;
    fetchedRef.current = true;

    async function load() {
      const cached = await loadJSON(CACHE_KEY); // fast path
      if (cached) {
        setPokemons(cached);
        return;
      }
      // fetch paginado + actualización progresiva...
      await saveJSON(CACHE_KEY, all); // persistir
    }
    load();
  }, []);
}
```

📌 **Buena práctica:** usa `useRef` para flags de control de efectos – a diferencia de `useState`, actualizar un ref no provoca un re-render.

Ventajas del patrón Provider + hook

Separar la carga de datos en un Provider y exponerla mediante un hook dedicado aporta beneficios concretos en rendimiento, mantenibilidad y experiencia de desarrollo.

Carga única

Los datos se cargan **una sola vez** al montar el Provider – ningún componente hijo repite la petición.

Acceso universal

Cualquier pantalla puede acceder a los datos llamando a `usePokemonContext()` sin prop drilling.

Cache persistente

El cache en `AsyncStorage` evita re-fetches entre sesiones, mejorando el tiempo de arranque.

StrictMode seguro

El `useRef` como flag evita el doble fetch que React StrictMode provoca en desarrollo.

Tour starter/useCustomHook.ts

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene una plantilla genérica con los tipos de entrada y salida marcados como `TODO`. Tu misión es adaptar esta plantilla al hook concreto que necesites para tu ejercicio.

```
// Plantilla genérica
export default function useCustomHook(
  _props: UseCustomHookProps
): UseCustomHookResult {
  // TODO: implementa el hook
  return {
    // TODO: devuelve los valores
  };
}
```

⚠ Adapta la plantilla al hook que necesites para tu ejercicio – no la uses directamente sin modificarla.

Solución — `solucion/usePokemonsContext.tsx`

La solución completa combina el `PokemonProvider` con el hook `usePokemonsContext`, incluyendo el guard de validación y el patrón `useRef` para evitar el doble fetch. Compara tu implementación con esta referencia.

```
export function usePokemonsContext() {
  const ctx = useContext(PokemonsContext);
  if (!ctx) throw new Error('Fuera de PokemonProvider');
  return ctx;
}

export function PokemonProvider({ children }) {
  const fetchedRef = useRef(false);
  const [pokemons, setPokemons] = useState([]);

  useEffect(() => {
    if (fetchedRef.current) return;
    fetchedRef.current = true;
    // fetch paginado + cache AsyncStorage
    load();
  }, []);

  return (
    <PokemonsContext.Provider value={{ pokemons }}>
      {children}
    </PokemonsContext.Provider>
  );
}
```

✓ Si cualquier pantalla puede llamar a `usePokemonsContext()` y recibe los datos sin repetir el fetch, ¡lo has conseguido! 🎉

Ejercicio T10

Pon en práctica el patrón Provider + hook. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. El bonus refuerza la extracción de lógica propia en un hook personalizado.

1 Lee la solución de referencia

Abre `solucion/usePokemonsContext.tsx` y léelo completo antes de empezar.

2 Crea el PokemonProvider

Añade el `PokemonProvider` en tu `App.tsx` envolviendo el árbol de navegación.

3 Consume el hook en tu pantalla

En tu pantalla de Pokédex usa `usePokemonsContext()` para acceder a los datos.

4 Bonus: useThemeMode()

Extrae tu lógica de tema (T04) en un hook personalizado `useThemeMode()`.

Resumen de la sesión

Estos son los conceptos clave que has trabajado hoy. Dominar este patrón te permitirá gestionar el estado global de forma limpia, eficiente y completamente libre de prop drilling en cualquier proyecto React Native.

Concepto	Clave	Para qué sirve
Hook personalizado	<code>function useXxx()</code> – empieza por use	Encapsula lógica stateful reutilizable
Cuándo crear	Lógica compleja o reutilizable	Evita duplicar código entre componentes
Context	<code>createContext</code> + <code>Provider</code>	Estado compartido entre componentes
<code>useRef</code> como flag	<code>fetchRef.current</code>	Evitar doble ejecución en StrictMode
<code>Provider</code> + hook	<code>usePokemonsContext()</code>	Patrón estándar para estado global

✅ ¡Enhorabuena! Ahora tienes las herramientas para extraer lógica compleja en hooks reutilizables y compartir estado global sin prop drilling en cualquier app React Native. 🚀